AULA PRÁTICA

Criação de tabelas e procedures em PL/SQL

Roteiro passo a passo para Oracle FreeSQL, com regras de negócio, tratamento de exceções e exercícios.

Duração sugerida: 90 a 120 minutos

Ambiente: Oracle FreeSQL - https://freesql.com/

Projeto: sistema acadêmico simplificado com alunos, cursos e matrículas

Orientação: execute cada bloco na ordem apresentada. Nas procedures, mantenha a barra / após o END para enviar a unidade PL/SQL para compilação.

1. Objetivos de aprendizagem

Ao final da aula, os estudantes deverão ser capazes de:

- criar tabelas com chaves primárias, estrangeiras e restrições;
- inserir e consultar registros;
- criar procedures com parâmetros de entrada;
- implementar regras de negócio com IF e RAISE_APPLICATION_ERROR;
- usar SELECT INTO e tratar exceções;
- executar procedures e verificar os resultados.

Cenário

A turma construirá um pequeno sistema de cursos. O banco terá alunos, cursos e matrículas. As procedures controlarão o cadastro, a disponibilidade de vagas e o lançamento de notas.

Modelo lógico

ALUNOS	MATRÍCULAS	CURSOS
aluno_id (PK) nome email (UQ) ativo data_cadastro	matricula_id (PK) aluno_id (FK) curso_id (FK) nota situacao	curso_id (PK) nome carga_horaria quantidade_vagas ativo

2. Criação das tabelas

2.1 Tabela ALUNOS

```
CREATE TABLE alunos (
    aluno_id       NUMBER GENERATED ALWAYS AS IDENTITY,
    nome           VARCHAR2(100) NOT NULL,
    email          VARCHAR2(150) NOT NULL,
    ativo          CHAR(1) DEFAULT 'S' NOT NULL,
    data_cadastro  DATE DEFAULT SYSDATE NOT NULL,

    CONSTRAINT pk_alunos PRIMARY KEY (aluno_id),
    CONSTRAINT uk_alunos_email UNIQUE (email),
    CONSTRAINT ck_alunos_ativo CHECK (ativo IN ('S', 'N'))
);
```

Ponto didático: IDENTITY gera o código automaticamente; UNIQUE impede e-mails repetidos; CHECK limita os valores aceitos.

2.2 Tabela CURSOS

```
CREATE TABLE cursos (
    curso_id          NUMBER GENERATED ALWAYS AS IDENTITY,
    nome              VARCHAR2(100) NOT NULL,
    carga_horaria     NUMBER(4) NOT NULL,
    quantidade_vagas  NUMBER(4) NOT NULL,
    ativo             CHAR(1) DEFAULT 'S' NOT NULL,

    CONSTRAINT pk_cursos PRIMARY KEY (curso_id),
    CONSTRAINT ck_cursos_carga CHECK (carga_horaria > 0),
    CONSTRAINT ck_cursos_vagas CHECK (quantidade_vagas > 0),
    CONSTRAINT ck_cursos_ativo CHECK (ativo IN ('S', 'N'))
);
```

2.3 Tabela MATRICULAS

```
CREATE TABLE matriculas (
    matricula_id   NUMBER GENERATED ALWAYS AS IDENTITY,
    aluno_id       NUMBER NOT NULL,
    curso_id       NUMBER NOT NULL,
    data_matricula DATE DEFAULT SYSDATE NOT NULL,
    nota           NUMBER(4,2),
    situacao       VARCHAR2(20) DEFAULT 'EM_CURSO' NOT NULL,

    CONSTRAINT pk_matriculas PRIMARY KEY (matricula_id),
    CONSTRAINT fk_matriculas_alunos
        FOREIGN KEY (aluno_id) REFERENCES alunos (aluno_id),
    CONSTRAINT fk_matriculas_cursos
        FOREIGN KEY (curso_id) REFERENCES cursos (curso_id),
    CONSTRAINT uk_aluno_curso UNIQUE (aluno_id, curso_id),
    CONSTRAINT ck_matriculas_nota CHECK (nota BETWEEN 0 AND 10),
    CONSTRAINT ck_matriculas_situacao
        CHECK (situacao IN
            ('EM_CURSO', 'APROVADO', 'REPROVADO', 'CANCELADO'))
);
```

Regra estrutural: a restrição UNIQUE (aluno_id, curso_id) impede a matrícula duplicada do mesmo aluno no mesmo curso.

3. Dados iniciais

Cadastre dois cursos para usar nos testes:

```
INSERT INTO cursos (nome, carga_horaria, quantidade_vagas)
VALUES ('Introdução ao SQL', 40, 3);

INSERT INTO cursos (nome, carga_horaria, quantidade_vagas)
VALUES ('Programação PL/SQL', 60, 2);
```

Confira os registros:

```
SELECT curso_id, nome, carga_horaria, quantidade_vagas, ativo
FROM cursos
ORDER BY curso_id;
```

4. Procedure para cadastrar aluno

Regras de negócio

- o nome deve possuir pelo menos três caracteres;
- o e-mail deve conter o caractere @;
- não pode haver dois alunos com o mesmo e-mail.

```
CREATE OR REPLACE PROCEDURE cadastrar_aluno (
    p_nome  IN VARCHAR2,
    p_email IN VARCHAR2
)
IS
    v_aluno_id alunos.aluno_id%TYPE;
BEGIN
    IF LENGTH(TRIM(p_nome)) < 3 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'O nome deve possuir pelo menos 3 caracteres.'
        );
    END IF;

    IF p_email IS NULL OR INSTR(p_email, '@') = 0 THEN
        RAISE_APPLICATION_ERROR(
            -20002,
            'Informe um e-mail válido.'
        );
    END IF;

    INSERT INTO alunos (nome, email)
    VALUES (TRIM(p_nome), LOWER(TRIM(p_email)))
    RETURNING aluno_id INTO v_aluno_id;

    DBMS_OUTPUT.PUT_LINE(
        'Aluno cadastrado. Código: ' || v_aluno_id
    );
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(
            -20003,
            'Já existe um aluno com esse e-mail.'
        );
END cadastrar_aluno;
/
```

Execução e testes

```
BEGIN
    cadastrar_aluno(
        p_nome  => 'Ana Silva',
        p_email => 'ana@escola.com'
    );
END;
/

BEGIN
    cadastrar_aluno(
        p_nome  => 'Bruno Souza',
        p_email => 'bruno@escola.com'
    );
END;
/
```

Consulte os alunos cadastrados:

```
SELECT aluno_id, nome, email, ativo, data_cadastro
FROM alunos
ORDER BY aluno_id;
```

Teste a restrição de e-mail duplicado:

```
BEGIN
    cadastrar_aluno(
        p_nome  => 'Outra Ana',
        p_email => 'ana@escola.com'
    );
END;
/
```

Resultado esperado: a execução deve produzir o erro personalizado: Já existe um aluno com esse e-mail.

5. Procedure para matricular aluno

Regras de negócio

- o aluno deve existir e estar ativo;
- o curso deve existir e estar ativo;
- o curso precisa ter vagas;
- o aluno não pode ser matriculado duas vezes no mesmo curso.

```
CREATE OR REPLACE PROCEDURE matricular_aluno (
    p_aluno_id IN NUMBER,
    p_curso_id IN NUMBER
)
IS
    v_aluno_ativo     alunos.ativo%TYPE;
    v_curso_ativo     cursos.ativo%TYPE;
    v_total_vagas     cursos.quantidade_vagas%TYPE;
    v_total_matriculas NUMBER;
    v_matricula_id    matriculas.matricula_id%TYPE;
BEGIN
    BEGIN
        SELECT ativo
        INTO v_aluno_ativo
        FROM alunos
        WHERE aluno_id = p_aluno_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20010, 'Aluno não encontrado.');
    END;

    IF v_aluno_ativo = 'N' THEN
        RAISE_APPLICATION_ERROR(-20011, 'O aluno está inativo.');
    END IF;

    BEGIN
        SELECT ativo, quantidade_vagas
        INTO v_curso_ativo, v_total_vagas
        FROM cursos
        WHERE curso_id = p_curso_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20012, 'Curso não encontrado.');
    END;

    IF v_curso_ativo = 'N' THEN
        RAISE_APPLICATION_ERROR(-20013, 'O curso está inativo.');
    END IF;

    SELECT COUNT(*)
    INTO v_total_matriculas
    FROM matriculas
    WHERE curso_id = p_curso_id
      AND situacao <> 'CANCELADO';

    IF v_total_matriculas >= v_total_vagas THEN
        RAISE_APPLICATION_ERROR(
            -20014,
            'Não existem vagas disponíveis.'
        );
    END IF;

    INSERT INTO matriculas (aluno_id, curso_id)
    VALUES (p_aluno_id, p_curso_id)
    RETURNING matricula_id INTO v_matricula_id;

    DBMS_OUTPUT.PUT_LINE(
        'Matrícula realizada. Código: ' || v_matricula_id
    );
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(
            -20015,
            'O aluno já está matriculado neste curso.'
        );
END matricular_aluno;
/
```

Execução

Primeiro, consulte os códigos gerados:

```
SELECT aluno_id, nome FROM alunos ORDER BY aluno_id;
SELECT curso_id, nome FROM cursos ORDER BY curso_id;
```

Depois, use os códigos encontrados. Neste exemplo, ambos são 1:

```
BEGIN
    matricular_aluno(
        p_aluno_id => 1,
        p_curso_id => 1
    );
END;
/
```

Confira a matrícula:

```
SELECT * FROM matriculas ORDER BY matricula_id;
```

6. Procedure para registrar nota

Regras de negócio

- a matrícula deve existir;
- a nota deve estar entre zero e dez;
- nota maior ou igual a 7 resulta em APROVADO;
- nota menor que 7 resulta em REPROVADO;
- uma matrícula cancelada não pode receber nota.

```
CREATE OR REPLACE PROCEDURE registrar_nota (
    p_matricula_id IN NUMBER,
    p_nota         IN NUMBER
)
IS
    v_situacao      matriculas.situacao%TYPE;
    v_nova_situacao matriculas.situacao%TYPE;
BEGIN
    IF p_nota IS NULL OR p_nota < 0 OR p_nota > 10 THEN
        RAISE_APPLICATION_ERROR(
            -20020,
            'A nota deve estar entre 0 e 10.'
        );
    END IF;

    BEGIN
        SELECT situacao
        INTO v_situacao
        FROM matriculas
        WHERE matricula_id = p_matricula_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20021, 'Matrícula não encontrada.');
    END;

    IF v_situacao = 'CANCELADO' THEN
        RAISE_APPLICATION_ERROR(
            -20022,
            'Uma matrícula cancelada não pode receber nota.'
        );
    END IF;

    IF p_nota >= 7 THEN
        v_nova_situacao := 'APROVADO';
    ELSE
        v_nova_situacao := 'REPROVADO';
    END IF;

    UPDATE matriculas
    SET nota = p_nota,
        situacao = v_nova_situacao
    WHERE matricula_id = p_matricula_id;

    DBMS_OUTPUT.PUT_LINE(
        'Nota registrada. Situação: ' || v_nova_situacao
    );
END registrar_nota;
/
```

Execução

```
BEGIN
    registrar_nota(
        p_matricula_id => 1,
        p_nota         => 8.5
    );
END;
/
```

7. Relatório final

```
SELECT
    m.matricula_id,
    a.nome AS aluno,
    c.nome AS curso,
    c.carga_horaria,
    m.data_matricula,
    m.nota,
    m.situacao
FROM matriculas m
JOIN alunos a ON a.aluno_id = m.aluno_id
JOIN cursos c ON c.curso_id = m.curso_id
ORDER BY m.matricula_id;
```

8. Controle de transação

Depois de conferir os resultados, confirme as alterações:

```
COMMIT;
```

Durante os testes, antes do COMMIT, as alterações também podem ser desfeitas:

```
ROLLBACK;
```

Boa prática: evite colocar COMMIT dentro das procedures. Assim, quem chama a procedure mantém o controle da transação.

9. Exercícios

- Tente cadastrar um aluno com nome de apenas duas letras.
- Tente cadastrar um e-mail sem @.
- Tente cadastrar dois alunos com o mesmo e-mail.
- Tente matricular o mesmo aluno duas vezes no mesmo curso.
- Preencha todas as vagas de um curso e tente realizar outra matrícula.
- Tente registrar uma nota maior que 10.
- Crie uma procedure chamada cancelar_matricula.
- Crie uma procedure chamada desativar_aluno.
- Acrescente frequência e exija nota mínima 7 e frequência mínima 75%.

10. Desafio complementar

Implemente a procedure abaixo como atividade final:

```
CREATE OR REPLACE PROCEDURE cancelar_matricula (
    p_matricula_id IN NUMBER
)
IS
    -- Declare aqui as variáveis necessárias.
BEGIN
    -- 1. Verifique se a matrícula existe.
    -- 2. Não permita cancelar uma matrícula já cancelada.
    -- 3. Atualize a situação para CANCELADO.
    -- 4. Mostre uma mensagem com DBMS_OUTPUT.PUT_LINE.
    -- 5. Trate a exceção NO_DATA_FOUND.
    NULL;
END cancelar_matricula;
/
```

Critérios de avaliação sugeridos

Critério	Pontos
Estrutura correta da procedure	2,0
Validação da matrícula	2,0
Aplicação da regra de negócio	2,0
Tratamento de exceções	2,0
Teste e demonstração do resultado	2,0

11. Encerramento

Revise com a turma a diferença entre restrições declarativas no banco e validações feitas na procedure. Destaque também o papel de parâmetros, variáveis locais, SELECT INTO, IF, exceções e transações.

Resultado esperado: ao concluir o roteiro, cada aluno terá criado um pequeno banco relacional e três procedures funcionais com regras de negócio.